Bug Algorithms

(Bug0, Bug1, Bug2, Tangent Bug)

What's Special About Bugs

- Many planning algorithms assume global knowledge
- Bug algorithms assume only *local* knowledge of the environment and a global goal
- Bug behaviors are simple:
  - 1) Follow a wall (right or left)
  - 2) Move in a straight line toward goal
- Bug 1 and Bug 2 assume essentially tactile sensing
- Tangent Bug deals with finite distance sensing

A Few General Concepts

- Workspace W
  - $\Re(2)$ or $\Re(3)$ depending on the robot
  - could be infinite (open) or bounded (compact)
- Obstacle WO_i
- Free workspace $W_{free} = W \setminus \cup WO_i$

Some notation:

q_{start} and q_{goal}

“hit point” q^H_i

„leave point“ q^L_i

A *path* is a sequence of hit/leave pairs bounded by q_{start} and q_{goal}

Assumptions

- known direction to goal
  - robot can measure distance d(x,y) between pts x and y
- otherwise local sensing
  - walls/obstacles & encoders
- *reasonable* world

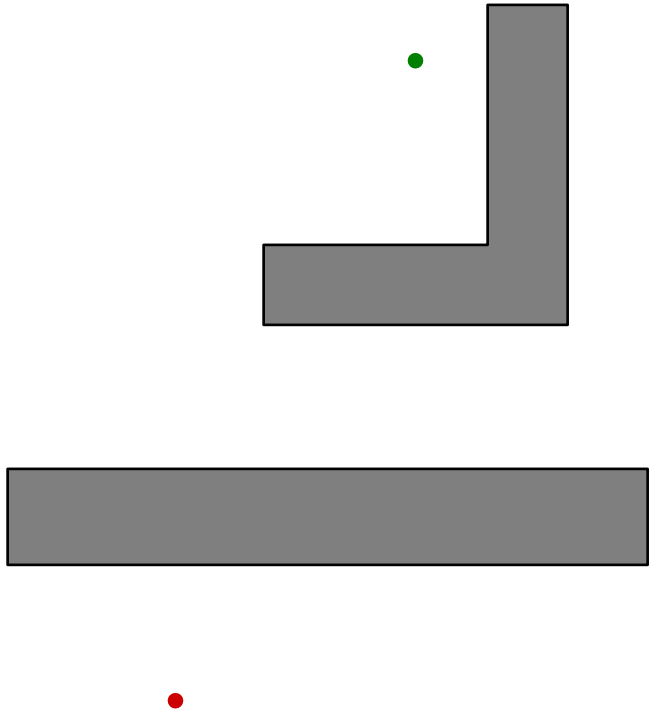
1) finitely many obstacles in any finite area

2) a line will intersect an obstacle finitely many times

3) Workspace is bounded

$W \subset B_r(x),\ r < \infty$

$B_r(x) = \{ y \in \mathfrak{R}(2) \mid d(x,y) < r \}$

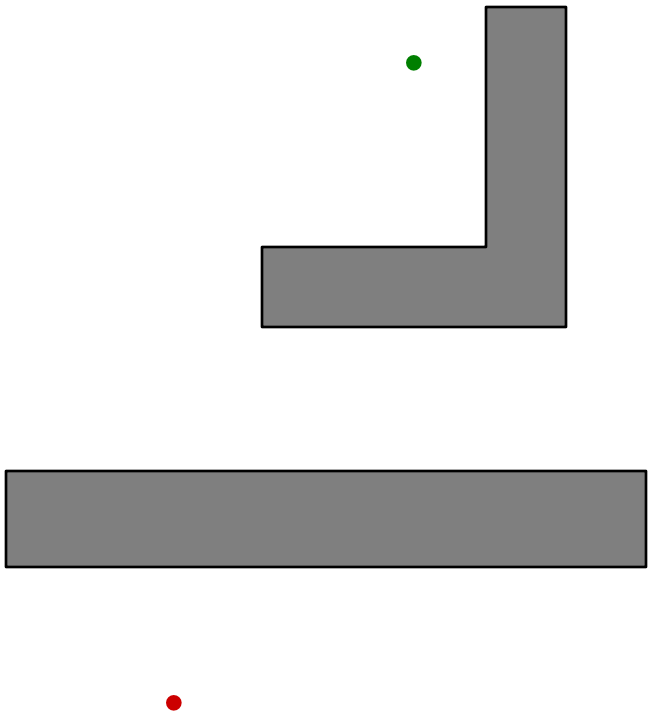


Beginner Strategy - Bug 0

"Bug 0" algorithm

- **known direction to goal**
- **otherwise local sensing**

walls/obstacles & encoders



1) head toward goal

2) follow obstacles until you can head toward the goal again

3) continue

path ?

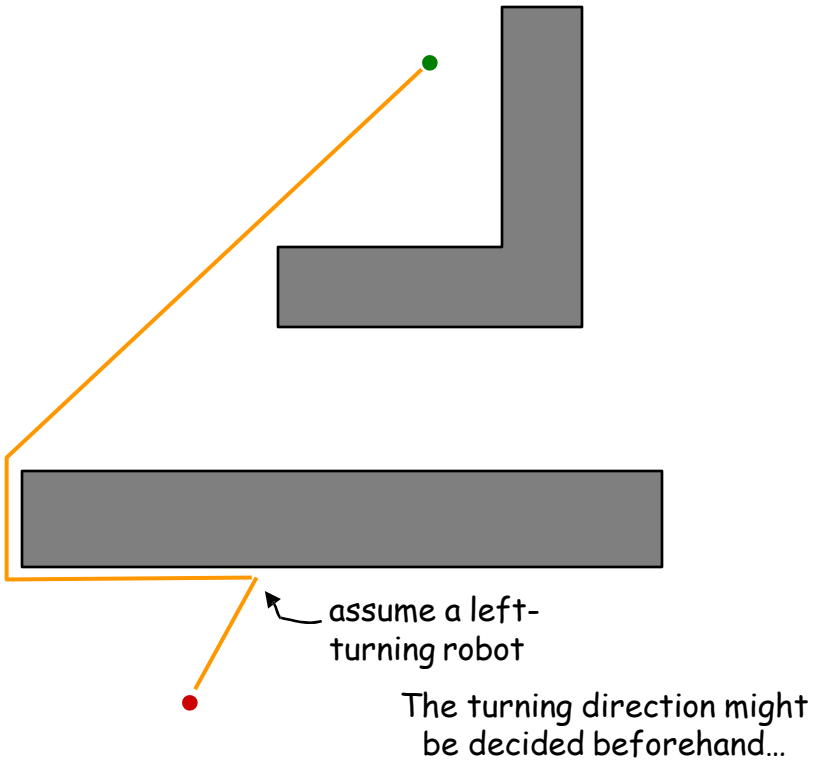
Beginner Strategy - Bug 0

Bug 0 algorithm

1)head toward goal

2)follow obstacles until you can head toward the goal again

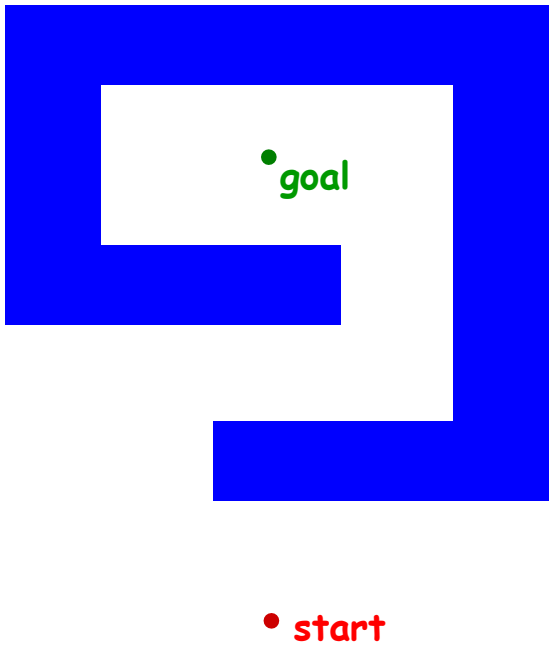
3) continue



OK ?

Beginner Strategy - Bug 0

What map will foil Bug 0 ?



"Bug 0" algorithm

1)head toward goal

2)follow obstacles until you can head toward the goal again

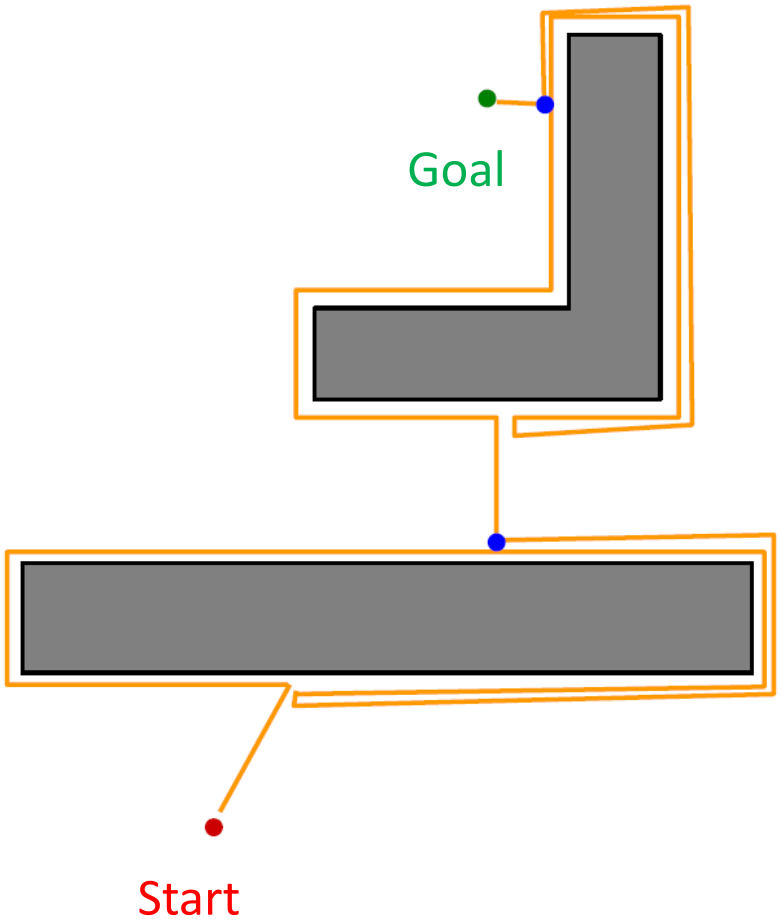
3) continue

Bug 1

But some computing power!

- known direction to goal
- otherwise local sensing

walls/obstacles & encoders



"Bug 1" algorithm

1) head toward goal

2) if an obstacle is encountered, circumnavigate it *and* remember how close you get to the goal

3) return to that closest point (by wall-following) and continue

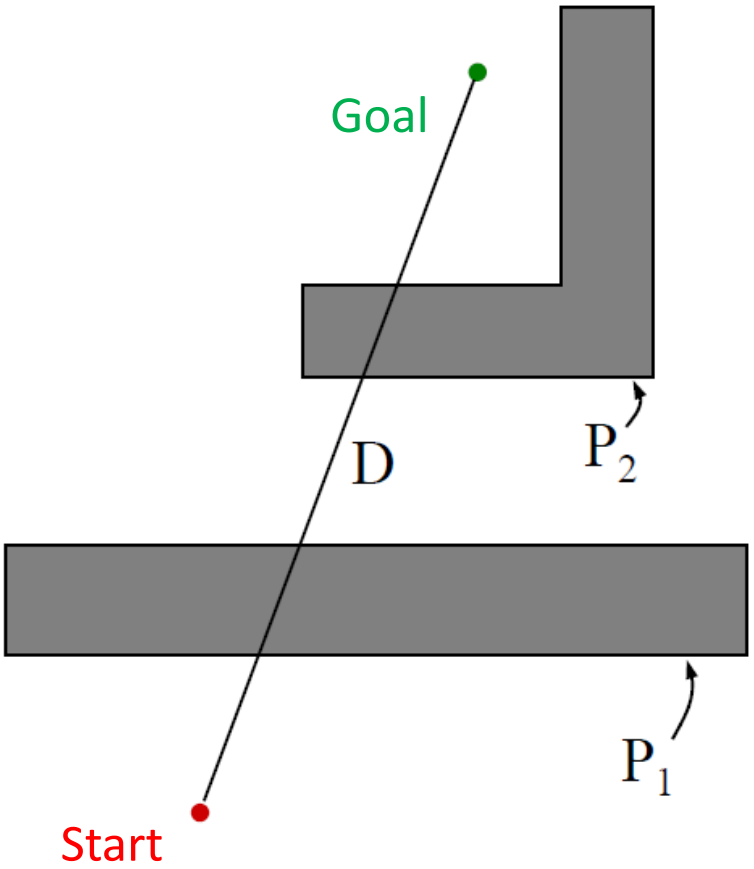
Bug 1 - more formally

- Let $q^L_0 = q_{start}$; i = 1
  repeat
  - repeat
    - from q^L_{i-1} move toward q_{goal}
  - until goal is reached or obstacle encountered at q^H_i
  - if goal is reached, exit
  - repeat
    - follow boundary recording pt q^L_i with shortest distance to goal
  - until q_{goal} is reached or q^H_i is re-encountered
  - if goal is reached, exit
  - Go to q^L_i
  - if move toward q_{goal} moves into obstacle
    - exit with failure
  - else
    - i=i+1
    - continue

Bug 1 analysis

Bug 1: Path Bounds

What are upper/lower bounds on the path length that the robot takes?



D = straight-line distance from start to goal

P_i = perimeter of the *i* th obstacle

Lower bound:

What's the shortest distance it might travel?

$$D$$

Upper bound:

What's the longest distance it might travel?

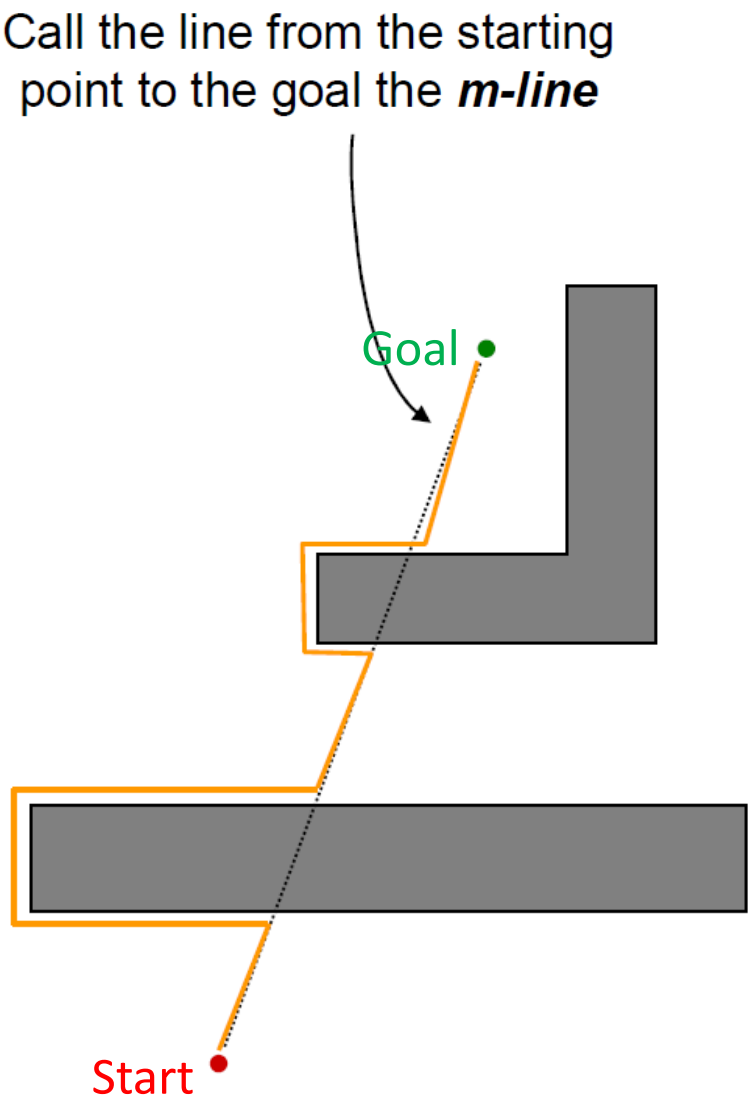
$$D + 1.5 \sum_i P_i$$

How Can We Show Completeness?

- An algorithm is *complete* if, in finite time, it finds a path if such a path exists or terminates with failure, if it does not.

- Suppose BUG1 were incomplete
  - Therefore, there is a path from start to goal
    - By assumption, it is finite length, and intersects obstacles a finite number of times.
  - BUG1 does not find it
    - Either it terminates incorrectly, or, it spends an infinite amount of time
    - Suppose it never terminates
      - but each leave point is closer to the obstacle than corresponding hit point
      - Each hit point is closer than the last leave point
      - Thus, there are a finite number of hit/leave pairs; after exhausting them, the robot will proceed to the goal and terminate
    - Suppose it terminates (incorrectly)
    - Then, the closest point after a hit must be a leave where it would have to move into the obstacle
      - But, the line from robot to goal must intersect object even number of times (Jordan curve theorem)
      - But then there is another intersection point on the boundary closer to object. Since we assumed there is a path, we must have crossed this pt on boundary which contradicts the definition of a leave point.

Bug 2



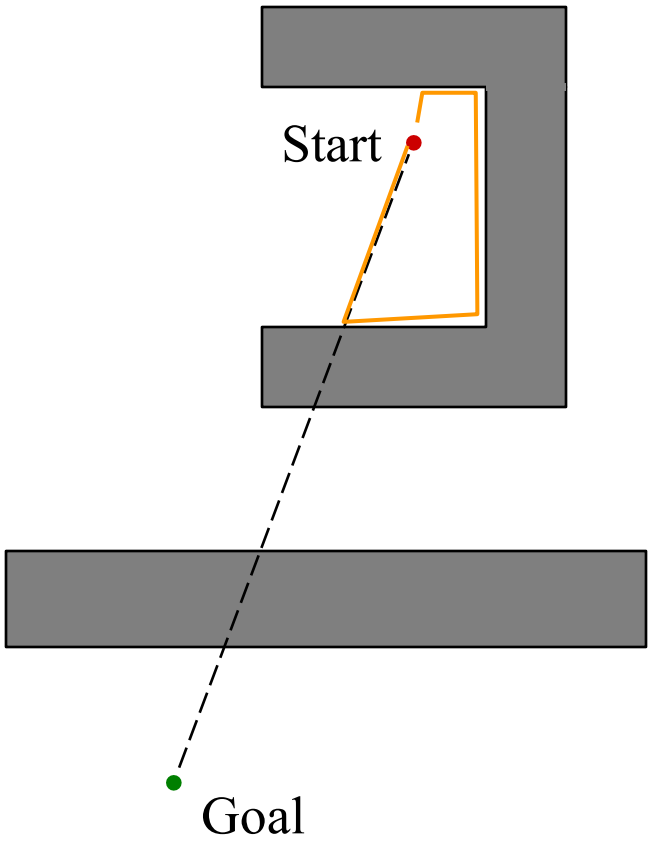
"Bug 2" Algorithm

1) head toward goal on the *m-line*

2) if an obstacle is in the way, follow it until you encounter the m-line again.

3) Leave the obstacle and continue toward the goal

Bug 2 – a better bug



"Bug 2" Algorithm

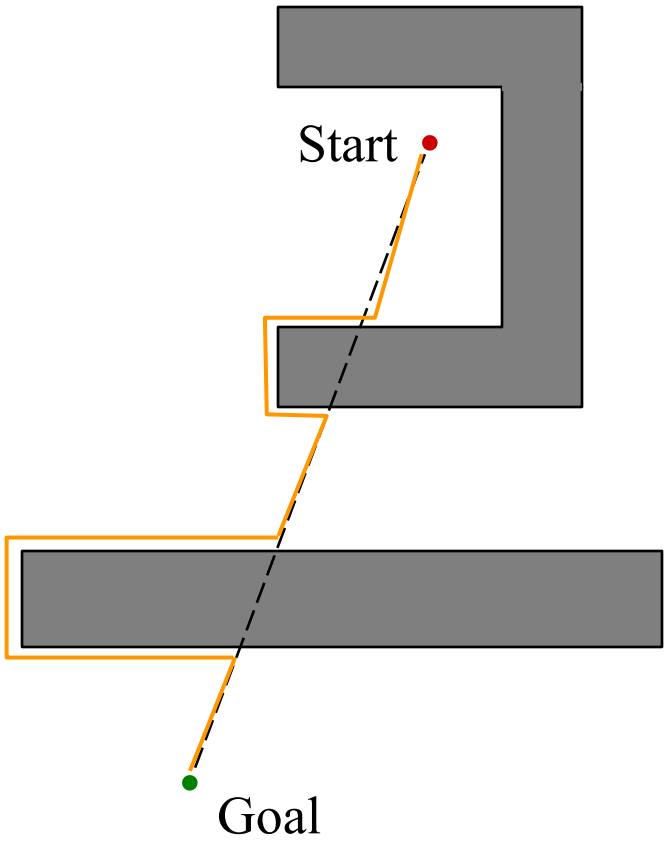
1)head toward goal on the *m-line*

2)if an obstacle is in the way, follow it until you encounter the m-line again.

3)Leave the obstacle and continue toward the goal

NO! How do we fix this?

Bug 2 – a better bug



"Bug 2" Algorithm

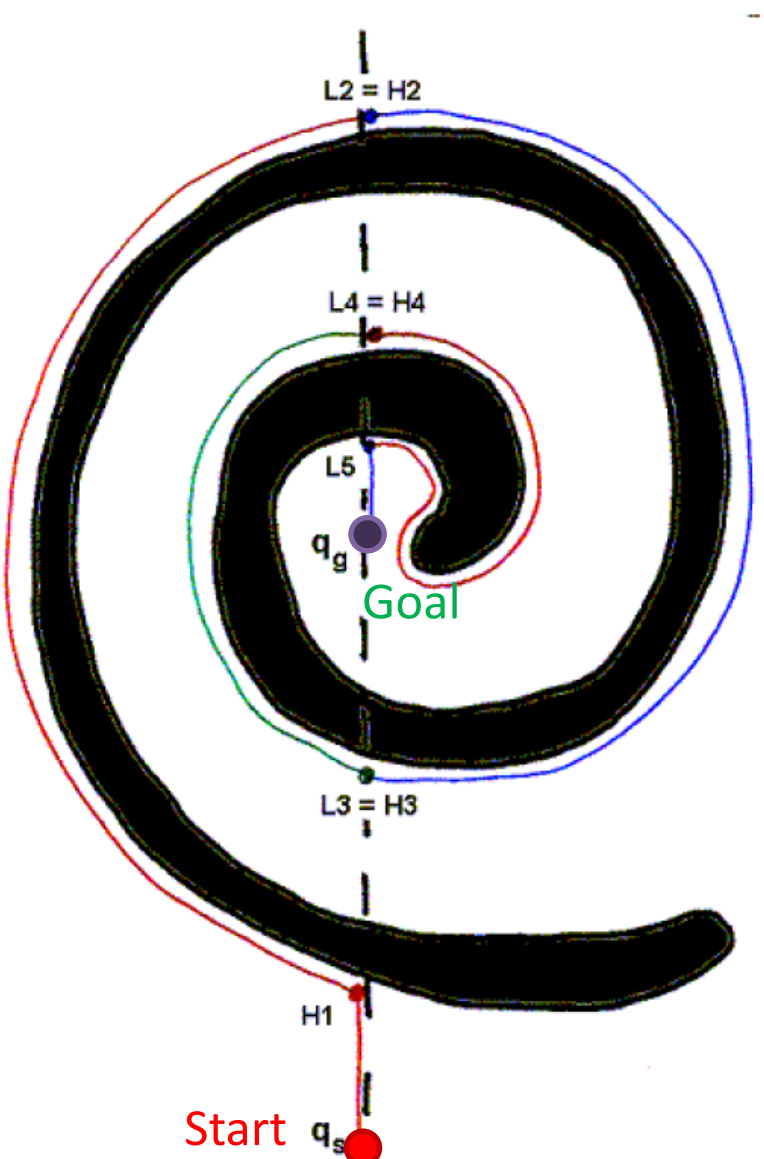
1)head toward goal on the *m-line*

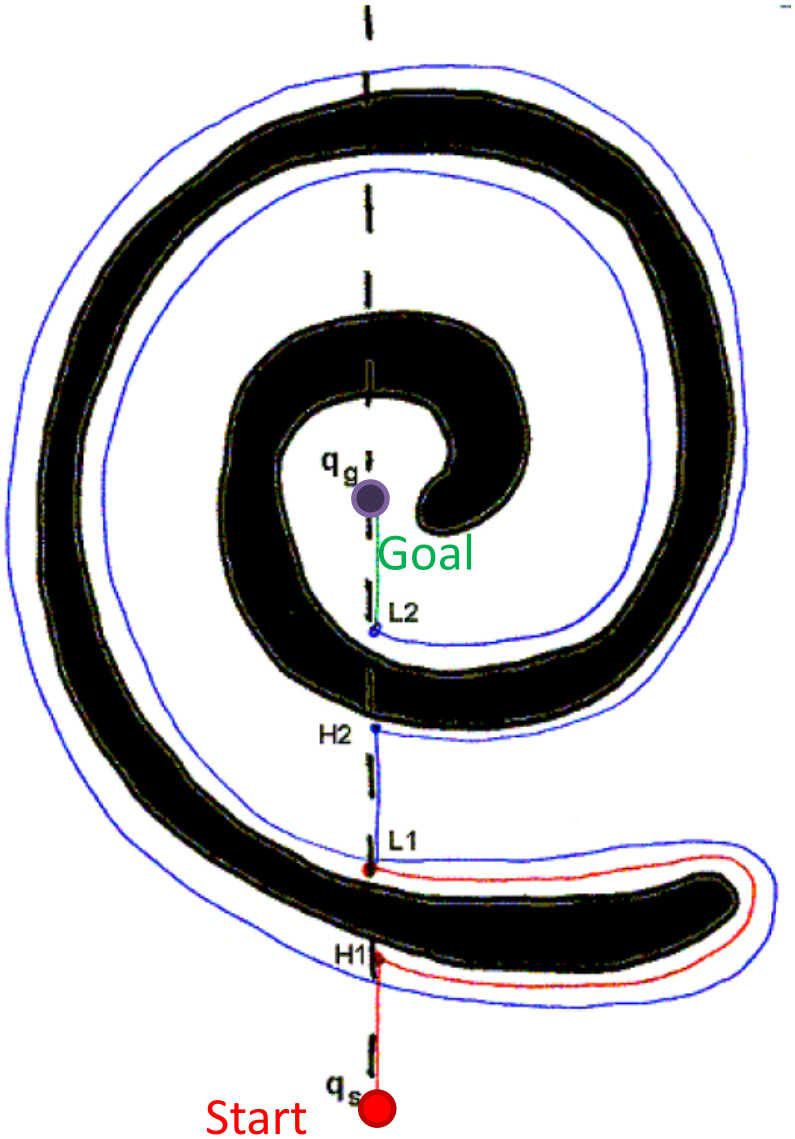
2)if an obstacle is in the way, follow it until you encounter the m-line again ***closer to the goal***.

3)Leave the obstacle and continue toward the goal

Better or worse than Bug1?

The Spiral


L2 = H2
L4 = H4
L5
q_g
Goal
L3 = H3
H1
Start
q_s


q_g
Goal
L2
H2
L1
H1
q_s
Start

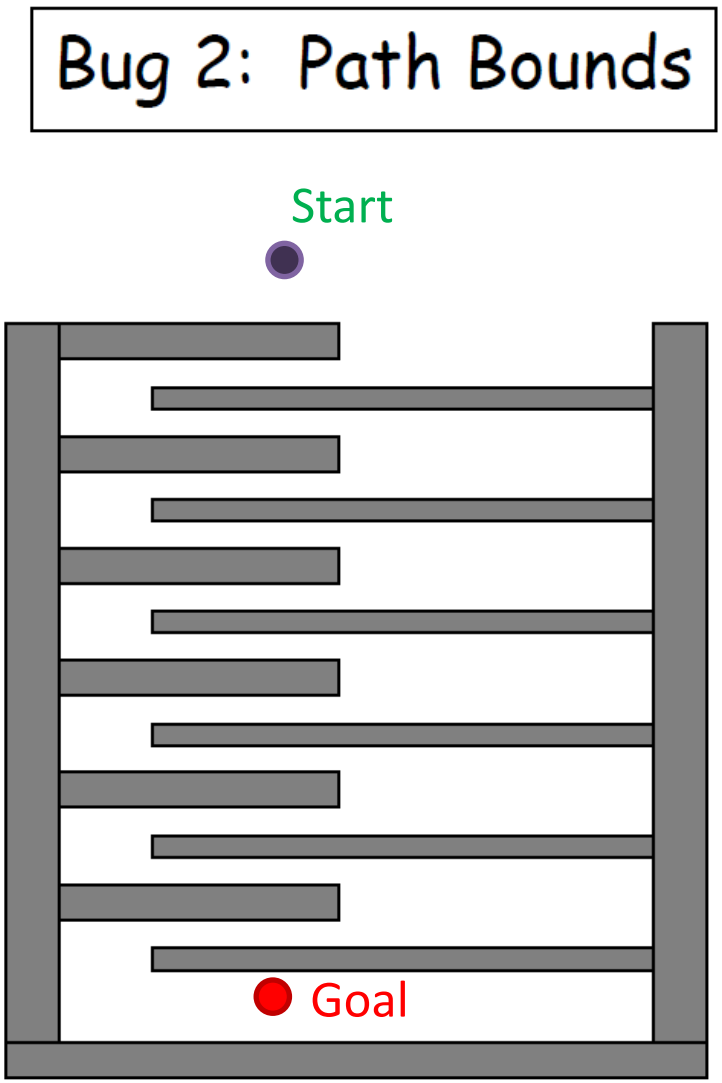
Bug 2 More formally

- Let $q^L_0 = q_{start;}$ i = 1

repeat

- repeat
  - from q^L_{i-1} move toward q_{goal} along the m-line
- until goal is reached or obstacle encountered at q^H_i
- if goal is reached, exit
- repeat
  - follow boundary
- until q_{goal} is reached or q^H_i is re-encountered or m-line is re-encountered, x is not q^H_i, $d(x,q_{goal}) < d(q^H_i,q_{goal})$ and way to goal is unimpeded
- if goal is reached, exit
- if q^H_i is reached, return failure
- else
  - $q^L_i = m$
  - i=i+1
  - continue

Bug 2 analysis



What are upper/lower bounds on the path length that the robot takes?

D = straight-line distance from start to goal

P_i = perimeter of the i th obstacle

Lower bound:

What's the shortest distance it might travel?

$$\mathbf{D}$$

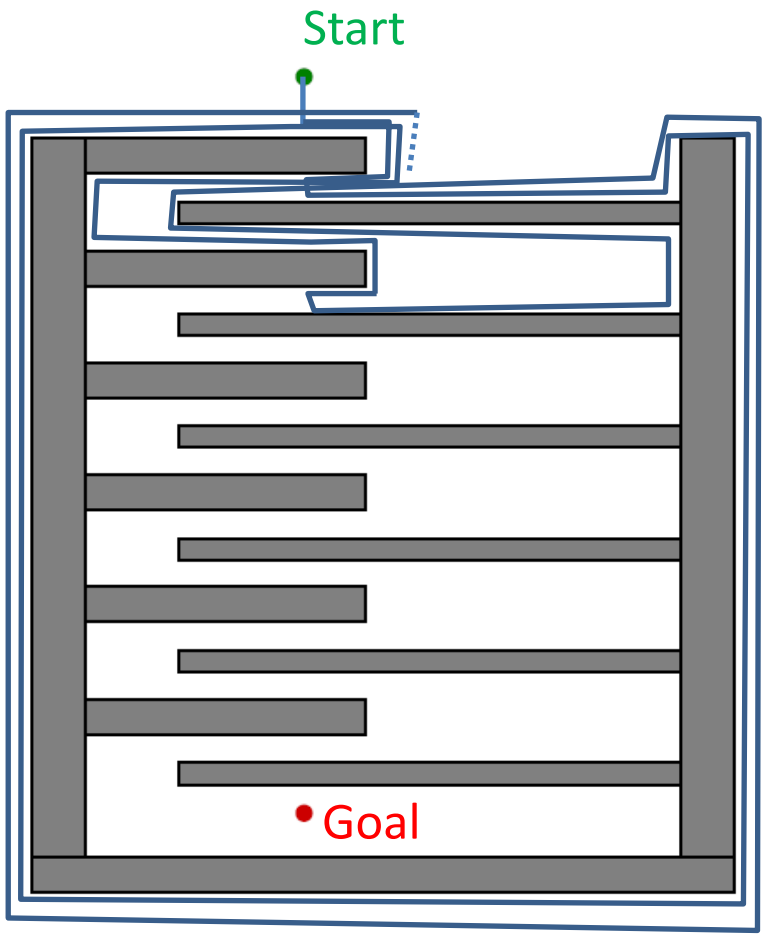
Upper bound:

What's the longest distance it might travel?

$$\mathbf{D + \sum_i \frac{n_i}{2} P_i}$$

$\mathbf{n_i}$ = # of s-line intersections of the i th obstacle

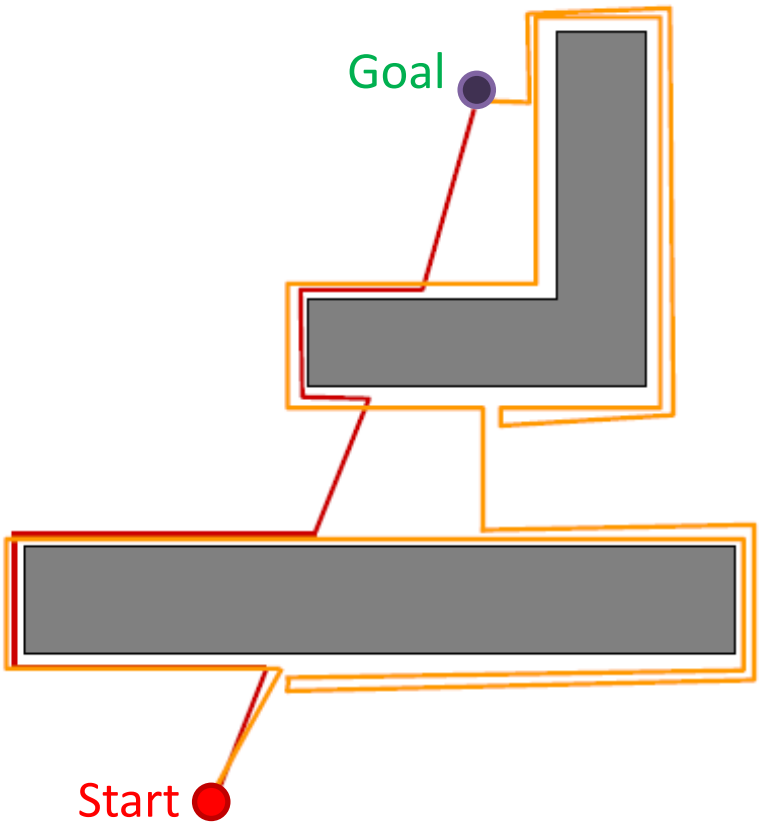
Bug 2 path example



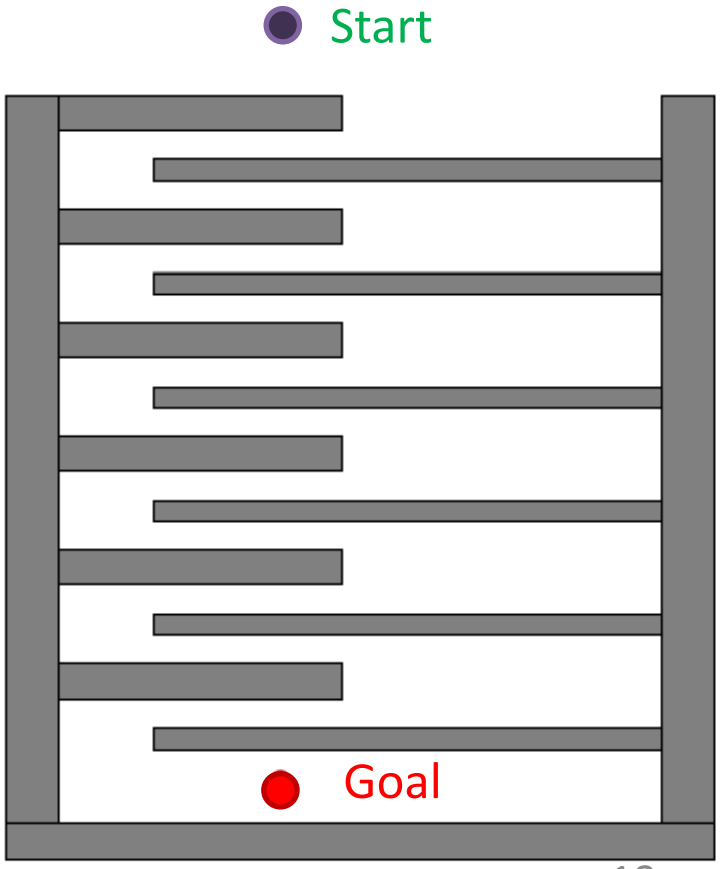
head-to-head comparison

Draw worlds in which Bug 2 does better than Bug 1 (and vice versa).

Bug 2 beats Bug 1



Bug 1 beats Bug 2



BUG 1 vs. BUG 2

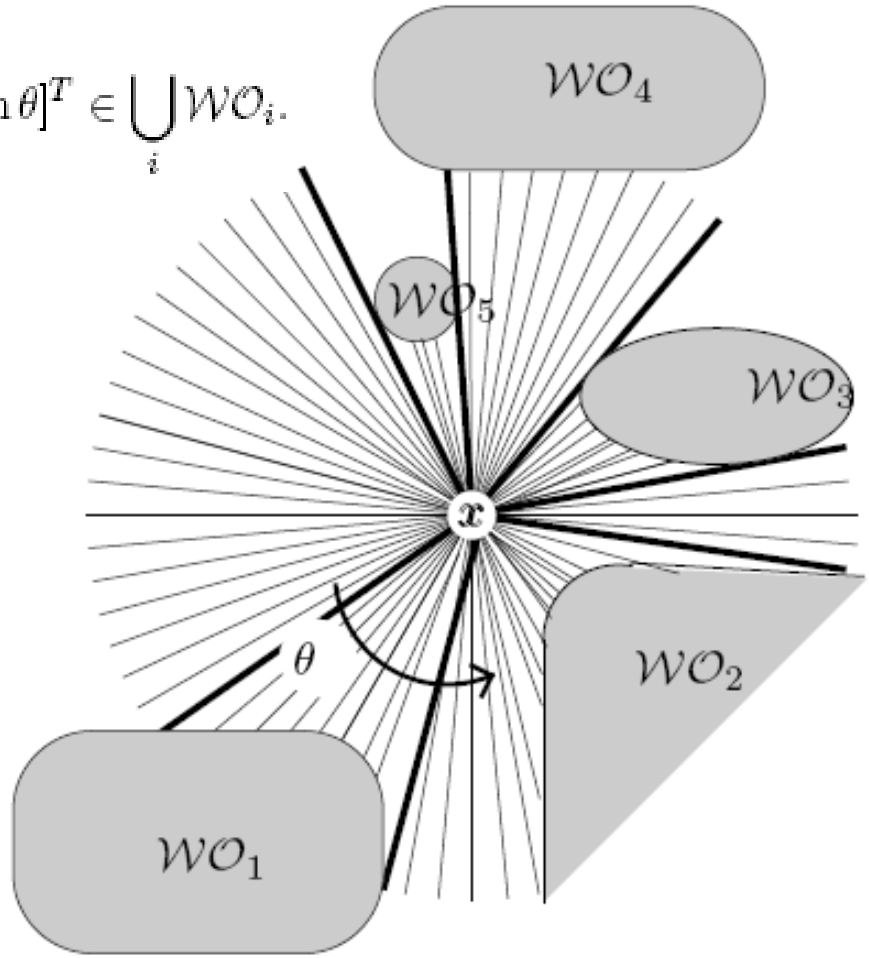
- BUG 1 is an *exhaustive search algorithm*
  - it looks at all choices before commiting
- BUG 2 is a *greedy* algorithm
  - it takes the first thing that looks better
- In many cases, BUG 2 will outperform BUG 1, but
- BUG 1 has a more predictable performance overall

Tangent BUG - Raw Distance Function

$$\rho(x,\theta) = \min_{\lambda \in [0,\infty]} d(x, x + \lambda[\cos\theta, \sin\theta]^T),$$

$$\text{such that} \quad x + \lambda[\cos\theta, \sin\theta]^T \in \bigcup_i \mathcal{WO}_i.$$

$$\rho : \mathbb{R}^2 \times S^1 \longrightarrow \mathbb{R}$$

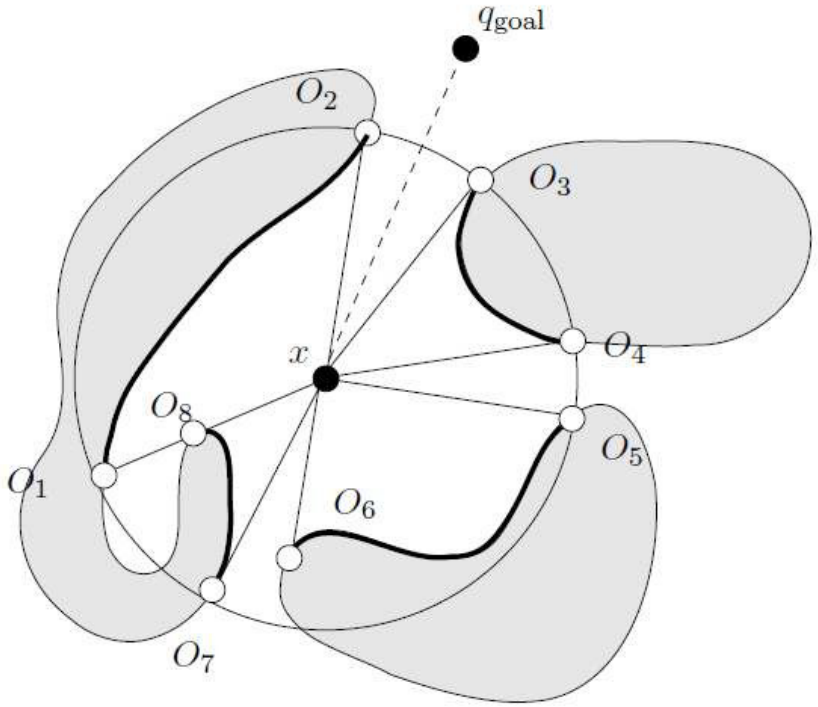


Saturated raw distance function

$$\rho_R(x,\theta) = \begin{cases} \rho(x,\theta), & \text{if } \rho(x,\theta) < R \\ \infty, & \text{otherwise.} \end{cases}$$

Intervals of Continuity

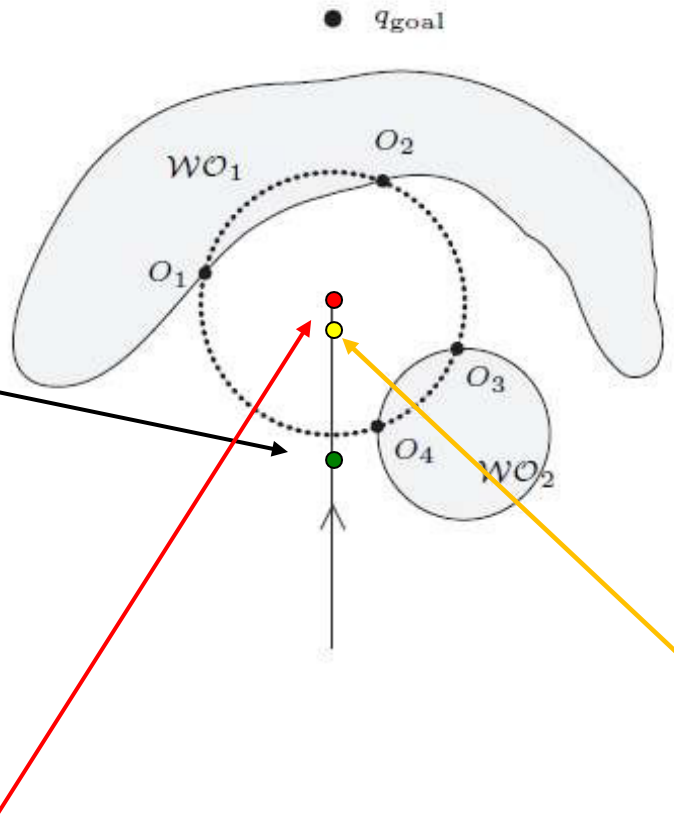
- Tangent Bug relies on finding endpoints of finite, continuous segments of ρ_R



- Concave (non-convex) obstactles might have more than 1 segment

Motion-to-Goal Transition from Moving Toward goal to "following obstacles"

Currently, the motion-to-goal behavior "thinks" the robot can get to the goal



Transition

Now, it starts to see something --- what to do?

Answer: For any O_i such that

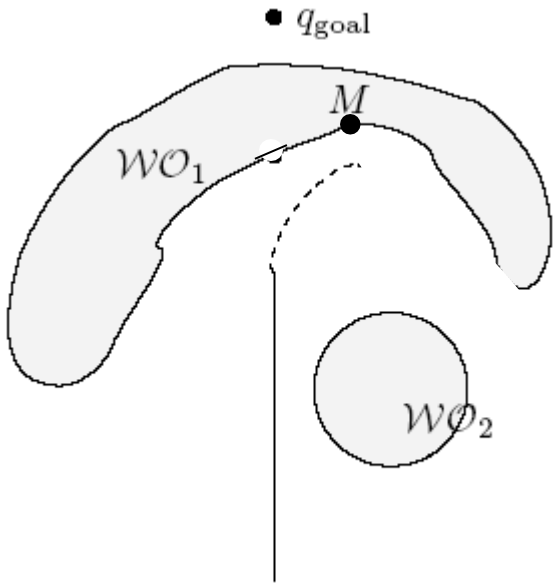
$d(O_i,q_{goal}) < d(x,q_{goal})$, choose the pt O_i that minimizes $d(x,O_i) + d(O_i,q_{goal})$

Transition *from* Motion-to-Goal

Choose the pt O_i that minimizes $d(x,O_i) + d(O_i,q_{goal})$

Problem: what if this distance starts to go up?

Answer: start to act like a BUG and follow boundary



M is the point on the "sensed" obstacle which has the shortest distance to the goal

Followed obstacle: the obstacle that we are currently sensing

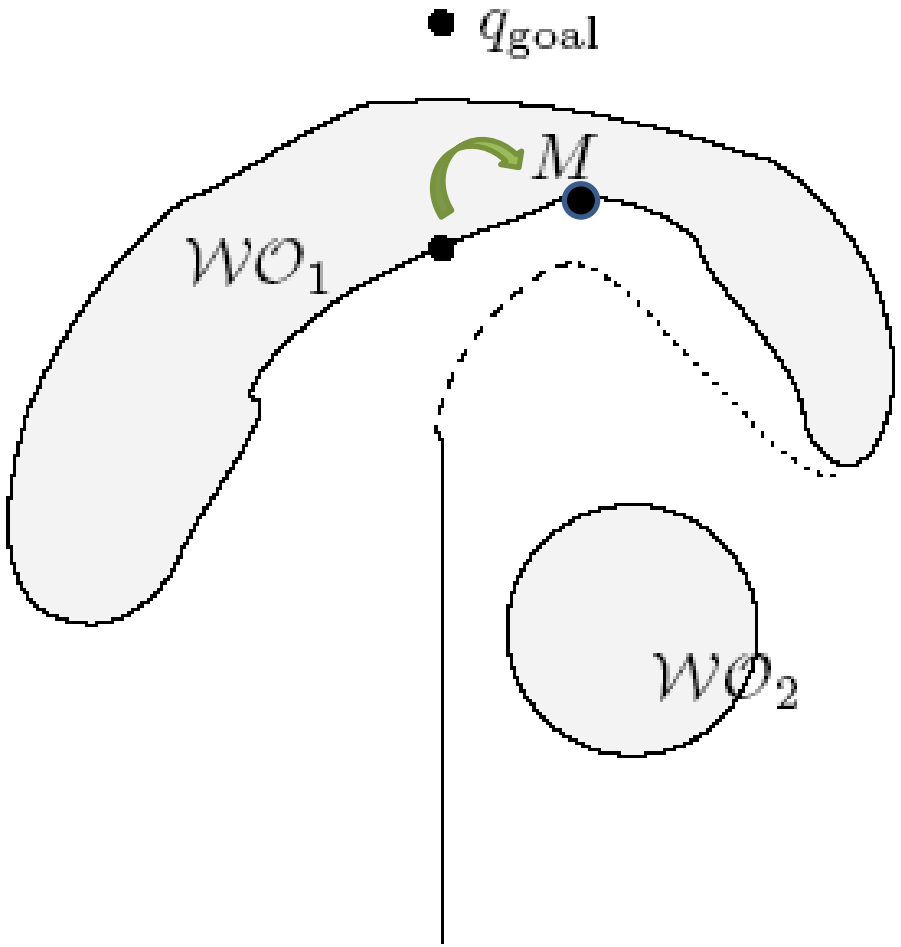
Blocking obstacle: the obstacle that intersects the segment

$$(1-\lambda)x + \lambda q_{goal} \quad \forall \lambda \in [0,1]$$

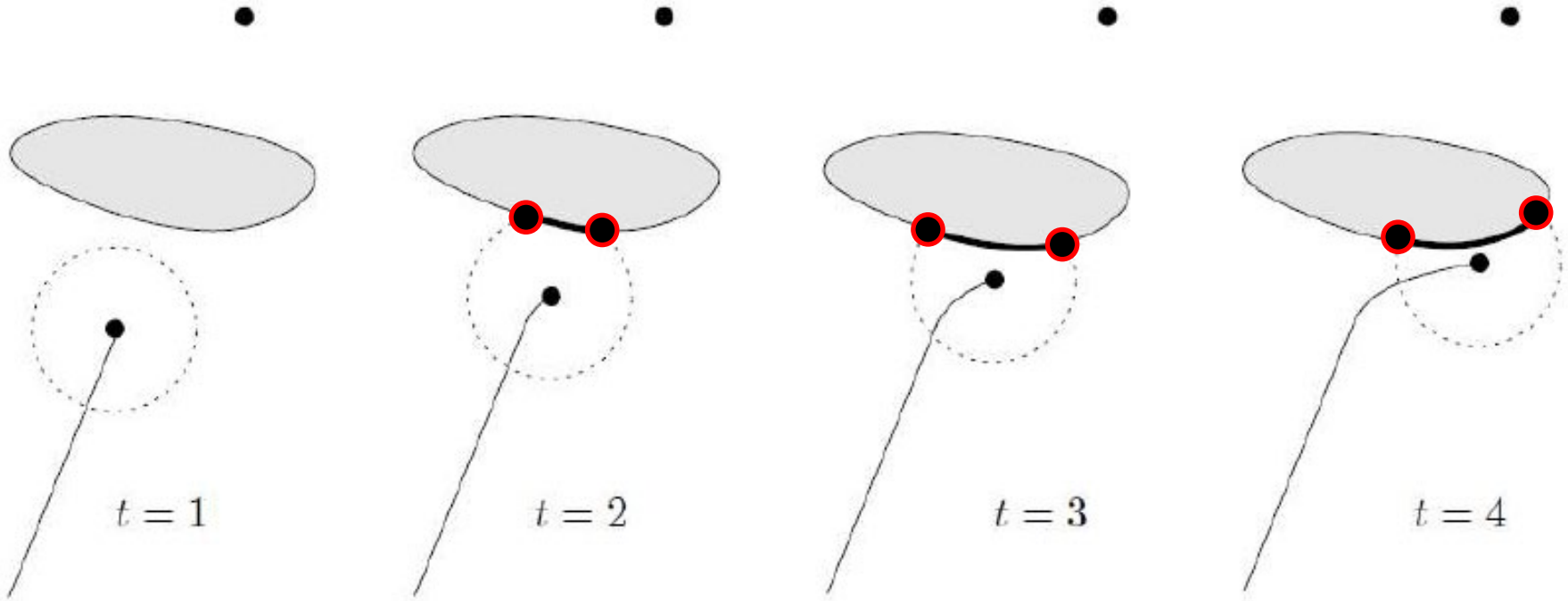
They start as the same

For any O_i such that $d(O_i,q_{goal}) < d(x,q_{goal})$,
choose the pt O_i that minimizes $d(x,O_i) + d(O_i,q_{goal})$

Transition ***from Motion-to-Goal***



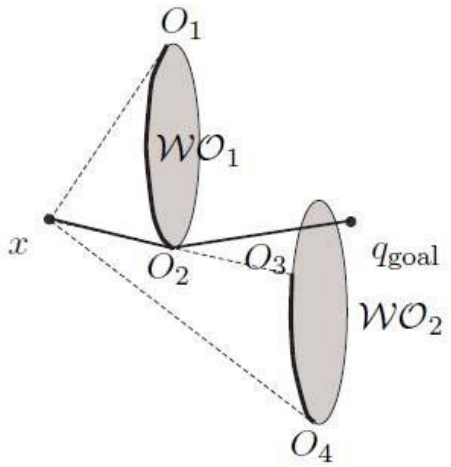
Motion To Goal Example



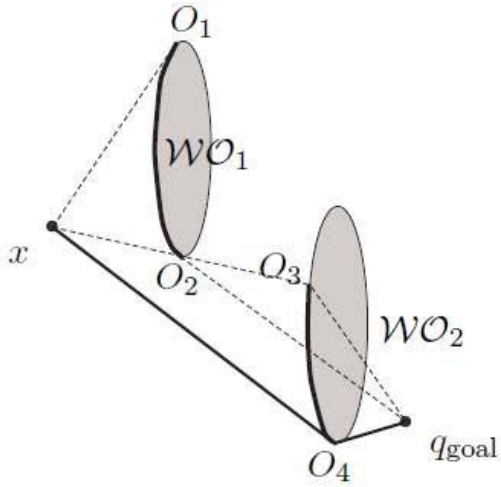
For any O_i such that $d(O_i,q_{goal}) < d(x,q_{goal})$,
choose the pt O_i that minimizes $d(x,O_i) + d(O_i,q_{goal})$

Minimize Heuristic Example

At x, robot knows only what it sees and where the goal is,



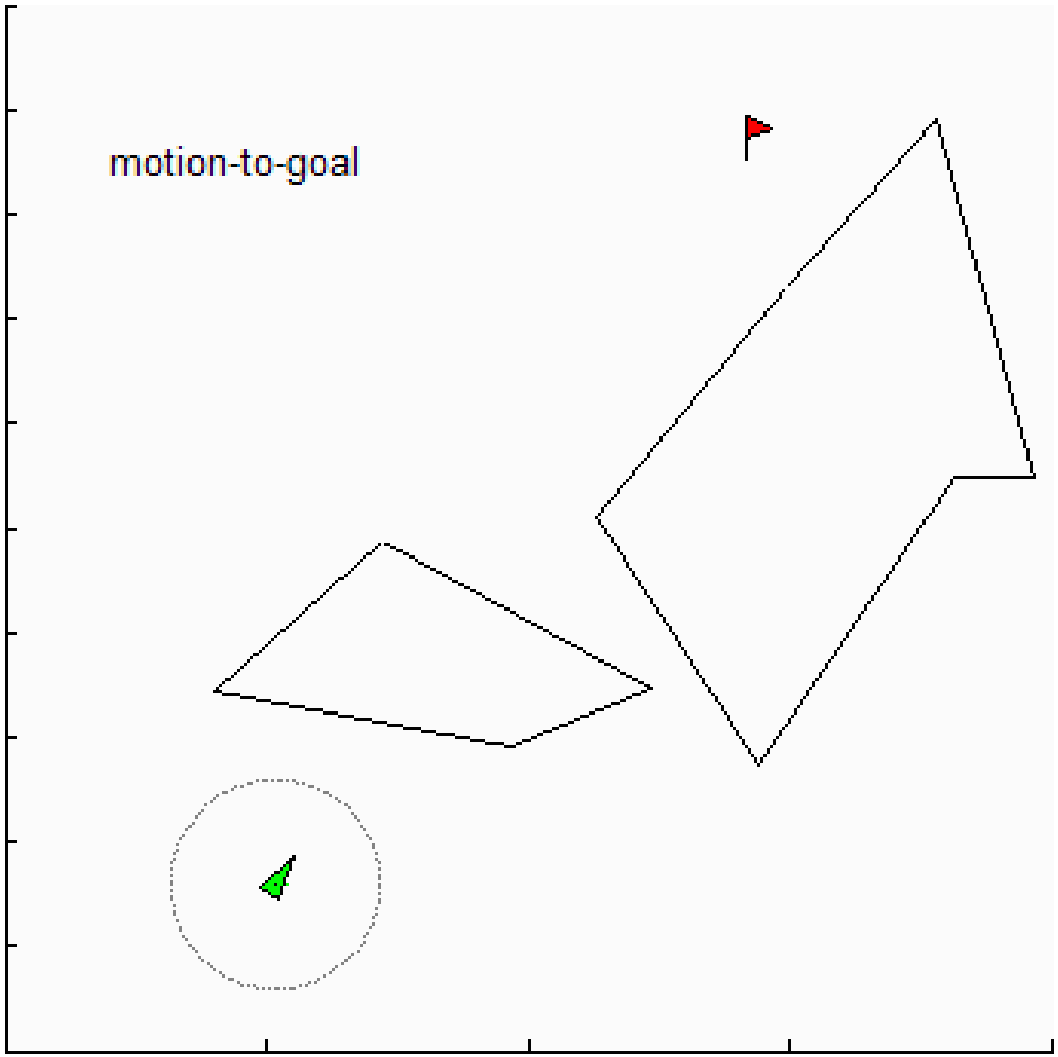
so moves toward O_2. Note the line connecting O_2 and goal pass through obstacle



so moves toward O_4. Note some "thinking" was involved and the line connecting O_4 and goal pass through obstacle

For any O_i such that $d(O_i,q_{goal}) < d(x,q_{goal})$,
choose the pt O_i that minimizes $d(x,O_i) + d(O_i,q_{goal})$

Motion To Goal Example

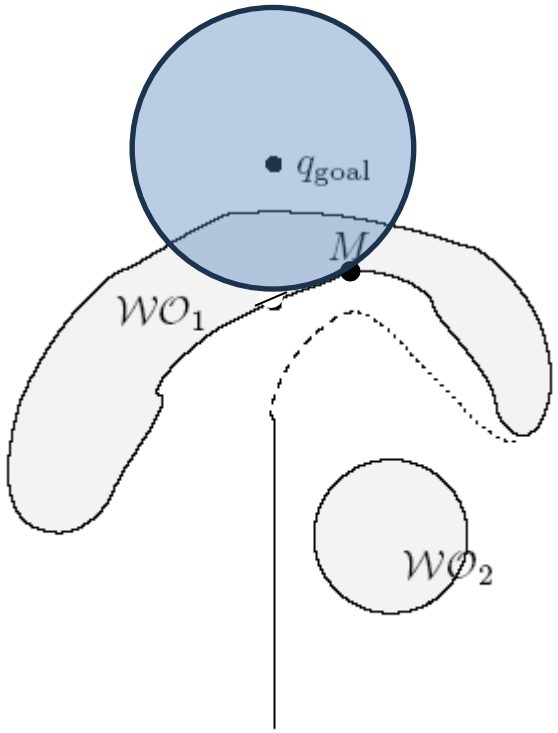


The Basic Ideas

- A motion-to-goal behavior as long as way is clear or there is a visible obstacle boundary pt that decreases heuristic distance
- A boundary following behavior invoked when heuristic distance increases.
- A value d_{min} which is the shortest distance observed thus far between the sensed boundary of the obstacle and the goal
- A value d_{leave} which is the shortest distance between any point in the currently sensed environment and the goal
- Terminate boundary following behavior when $d_{leave} < d_{min}$

Boundary Following

Move toward the O_i on the followed obstacle in the “chosen” direction



M is the point on the “sensed” obstacle which has the shorted distance to the goal

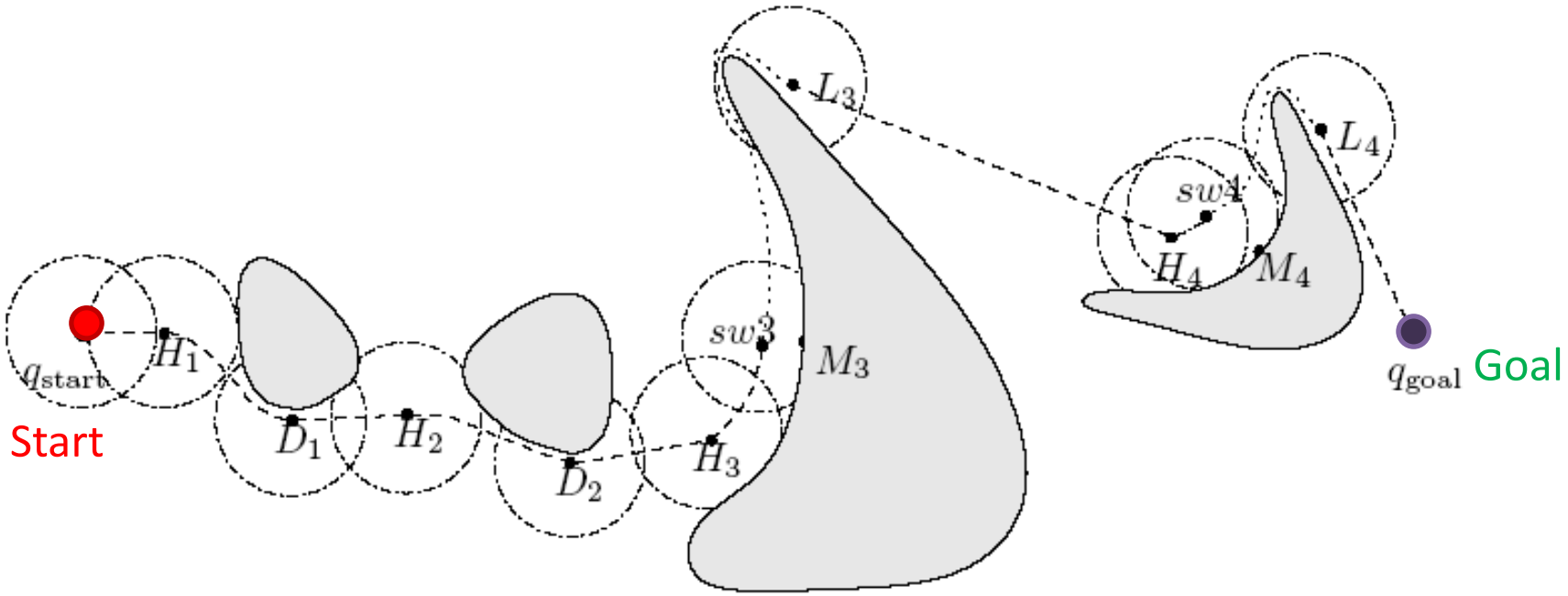
Followed obstacle: the obstacle that we are currently sensing

Blocking obstacle: the obstacle that intersects the segment

Maintain d_{min} and d_{leave}

They start as the same

Tangent Bug Algorithm



H : hit point
M : minimum point
D : depart point
L : leave point

Terminate boundary following behavior when $d_{leave} < d_{min}$

d_{min} and d_{leave}

- d_{min} is the shortest distance, observed thus far, between the sensed boundary of the obstacle and the goal

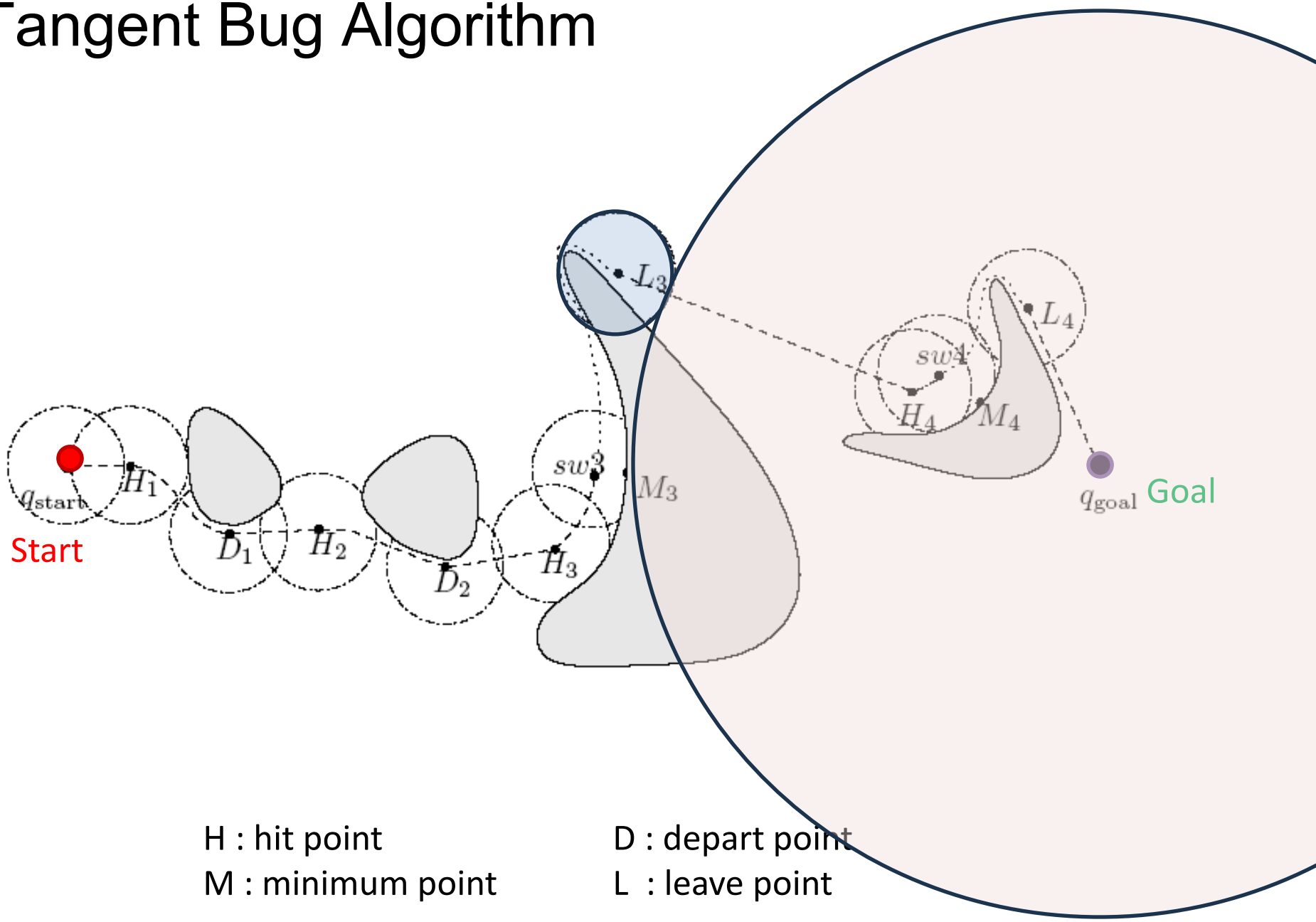
- d_{leave} is the shortest distance between any point in the currently sensed environment and the goal

$$V_R(x) = \{y \in W_{free} \mid d(x,y) < R \text{ and } \lambda x + (1-\lambda) y \in W_{free} \text{ for all } \lambda \in [0,1]\}$$

$$d_{leave}(x) = \min_{y \in V(x)} d(goal, y)$$

- Terminate boundary following behavior when $d_{leave} < d_{min}$
- Initialize with $x = q_{start}$ and $d_{leave} = d(q_{start}, q_{goal})$

Tangent Bug Algorithm



H : hit point
M : minimum point
D : depart point
L : leave point

Terminate boundary following behavior when $d_{leave} < d_{min}$

Tangent Bug Algorithm

1) repeat
   a) Compute continuous range segments in view
   b) Move toward $n \in \{T, O_i\}$ that minimizes $h(x,n) = d(x,n) + d(n,q_{goal})$

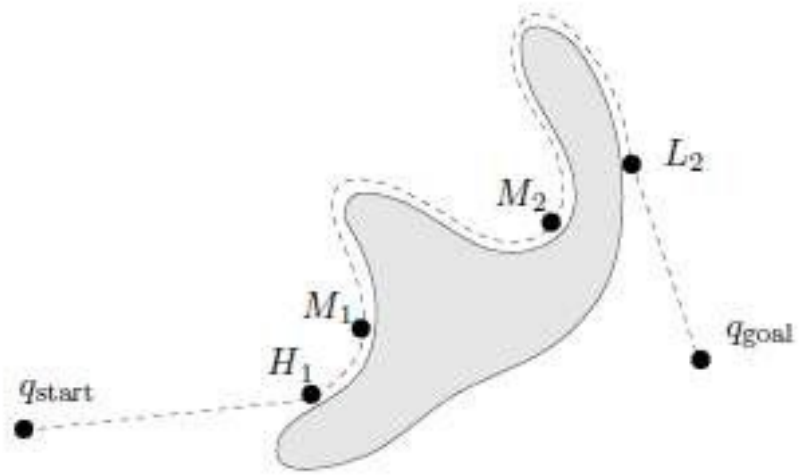
   until
   a) goal is encountered, or
   b) the value of h(x,n) begins to increase

1) follow boundary continuing in same direction as before repeating
   a) update $\{O_i\}$, d_{leave} and d_{min}

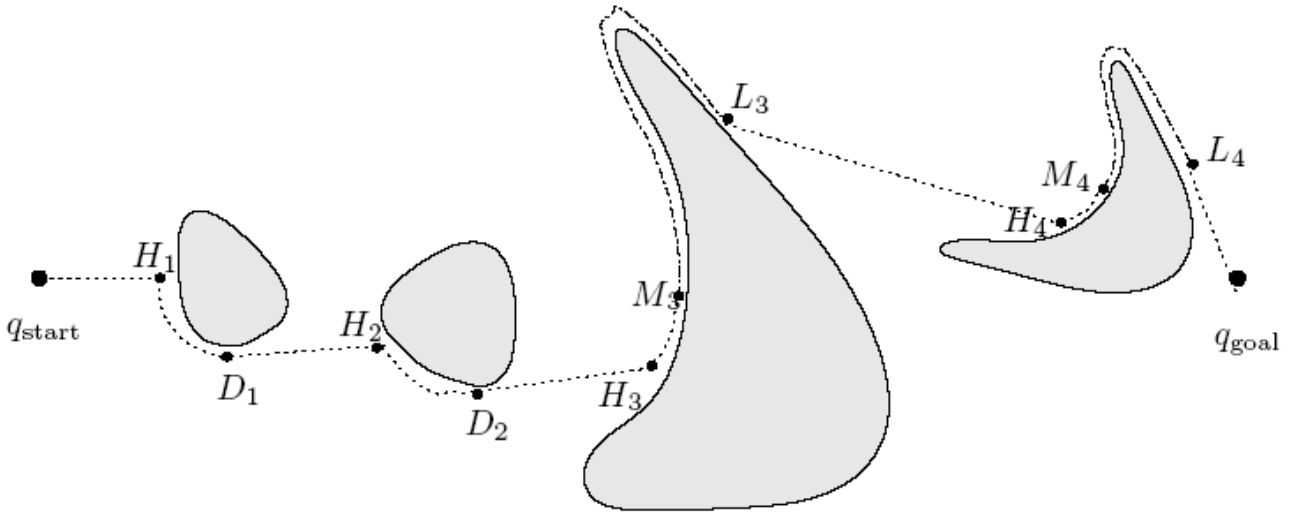
   until
   a) goal is reached
   b) a complete cycle is performed (goal is unreachable)
   c) $d_{leave} < d_{min}$

Note the same general proof reasoning as before applies, although the definition of hit and leave points is a little trickier.

d_{min} is constantly updated

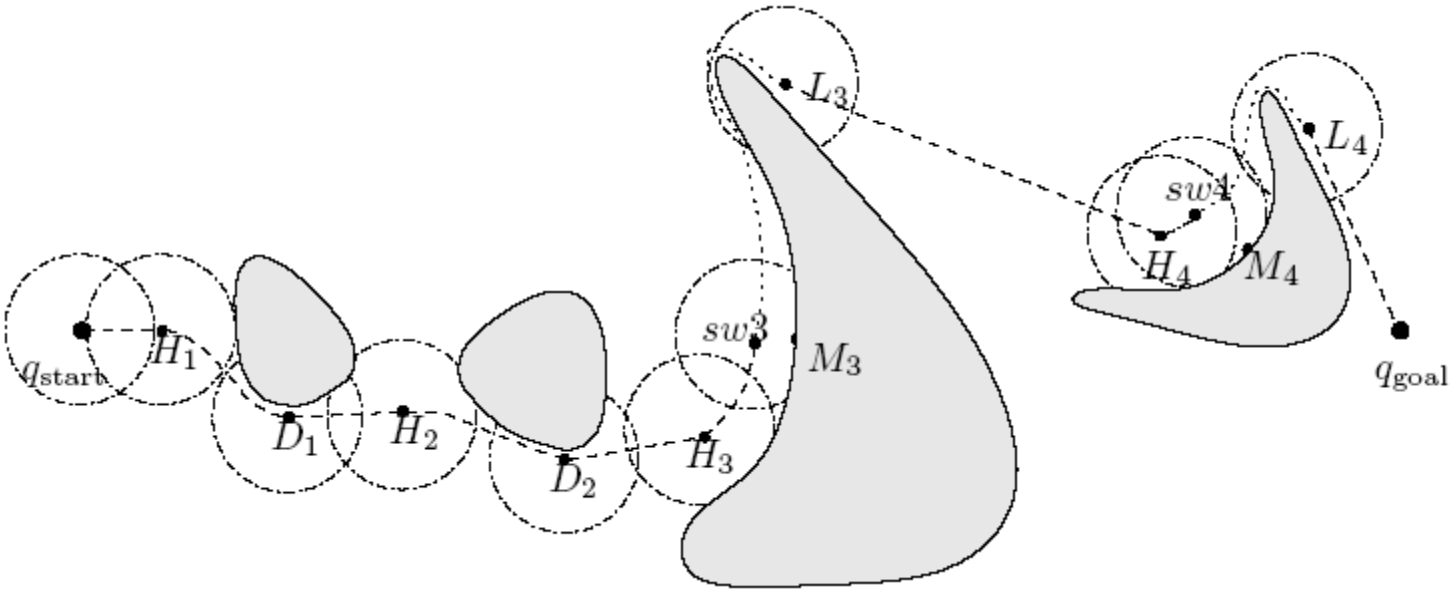


Example: Zero Senor Range

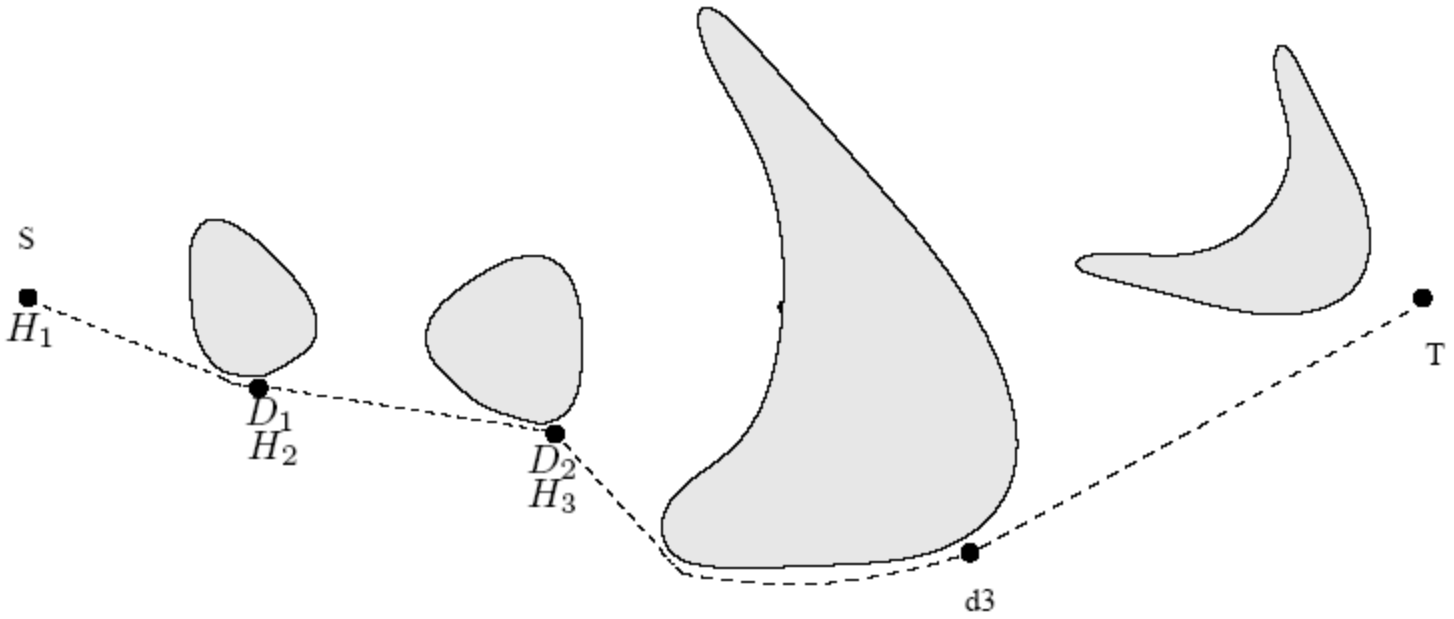


1. Robot moves toward goal until it hits obstacle 1 at H1
2. Pretend there is an infinitely small sensor range and the Oi which minimizes the heuristic is to the right
3. Keep following obstacle until robot can go toward obstacle again
4. Same situation with second obstacle
5. At third obstacle, the robot turned left until it could not increase heuristic
6. d_{min} is distance between M_3 and goal, d_{leave} is distance between robot and goal because sensing distance is zero

Example: Finite Sensor Range

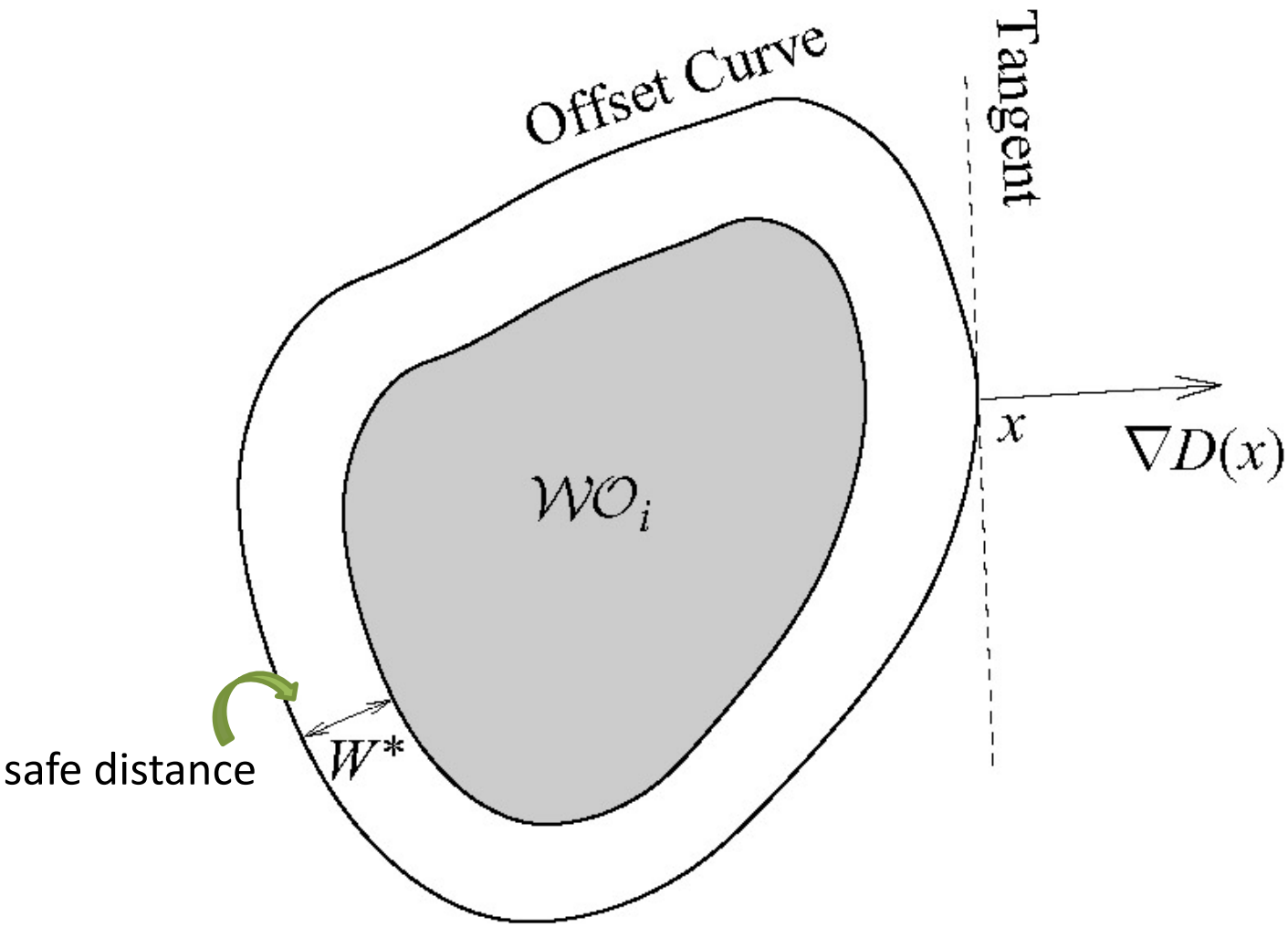


Example: Infinite Sensor Range



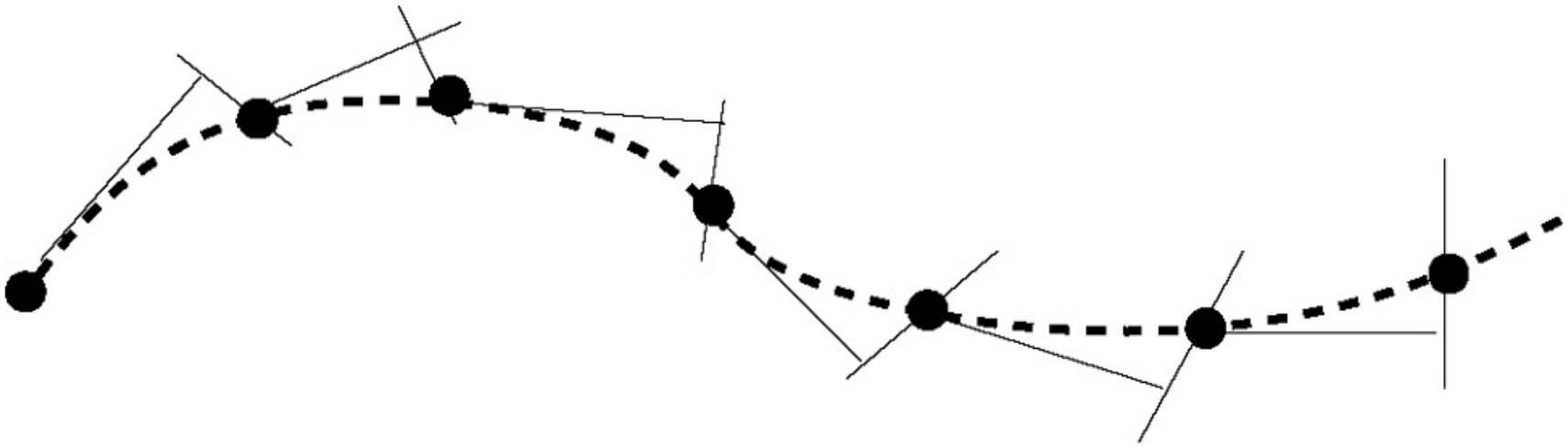
Implementation

What Information: The Tangent Line



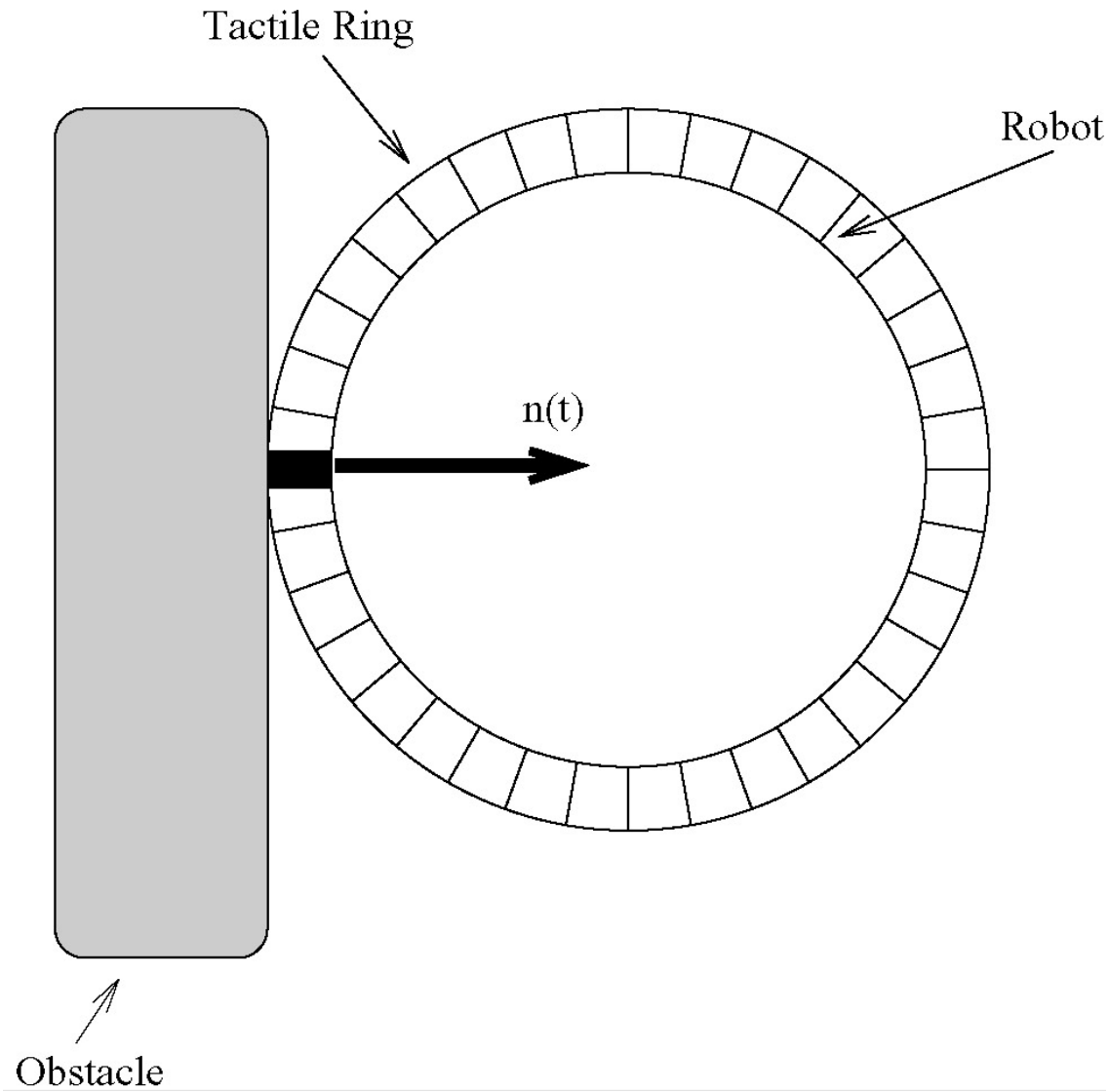
The dashed line represents the tangent to the offset curve at x.

How to Process Sensor Information



The dashed line is the actual path, but the robot follows the thin black lines, predicting and correcting along the path. The black circles are samples along the path.

Sensors



Tactile sensors

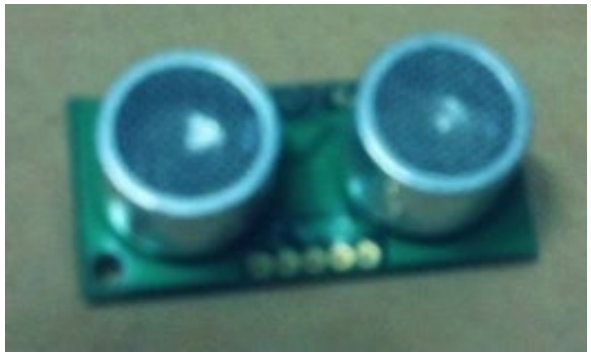
Tactile sensors are employed wherever interactions between a contact surface and the environment are to be measured and registered.

A tactile sensor is a device which receives and responds to a signal or stimulus having to do with force.

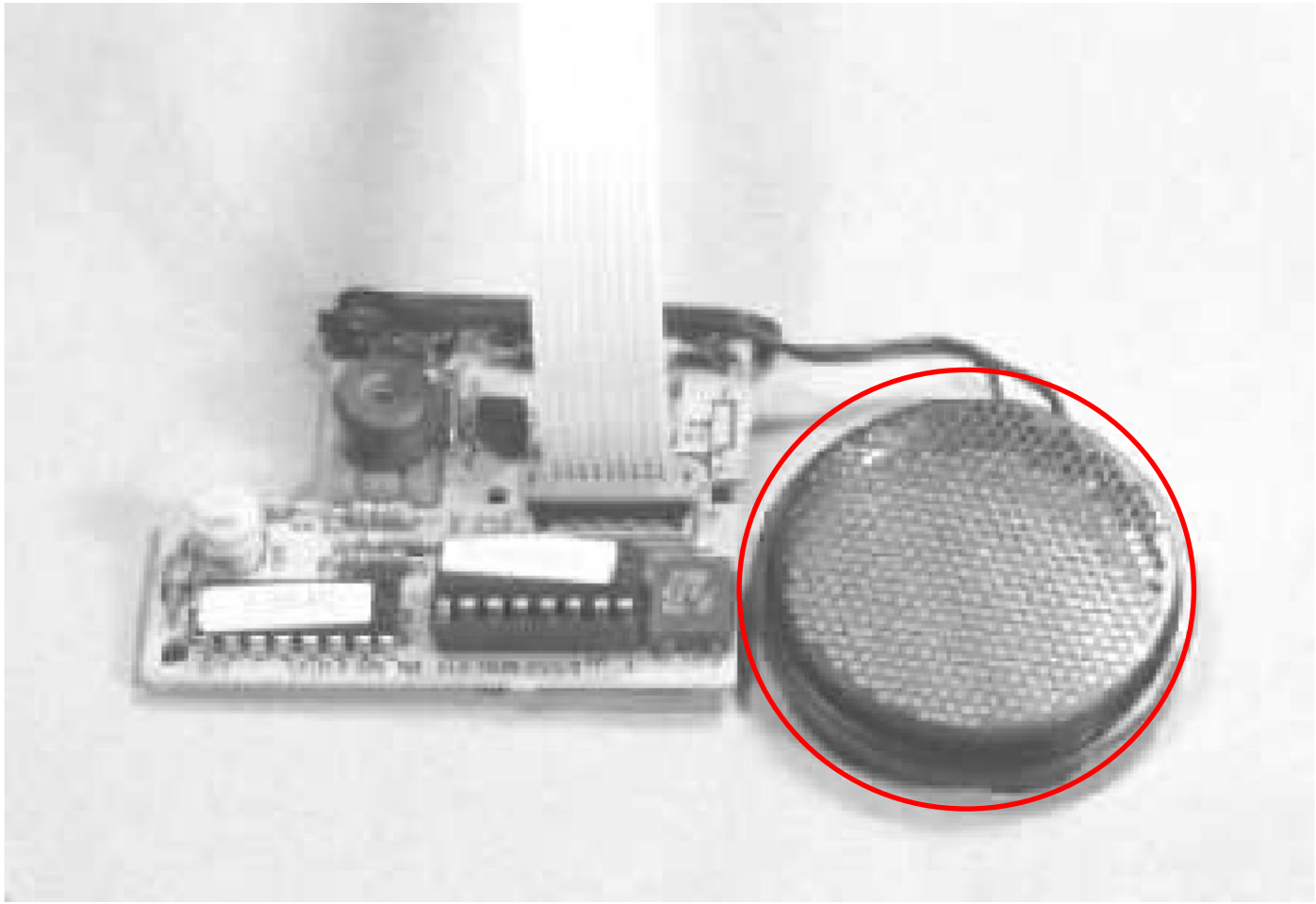
<daVinci medical system>

Ultrasonic sensors

Ultrasonic sensors generate high frequency sound waves and evaluate the echo which is received back by the sensor. Sensors calculate the time

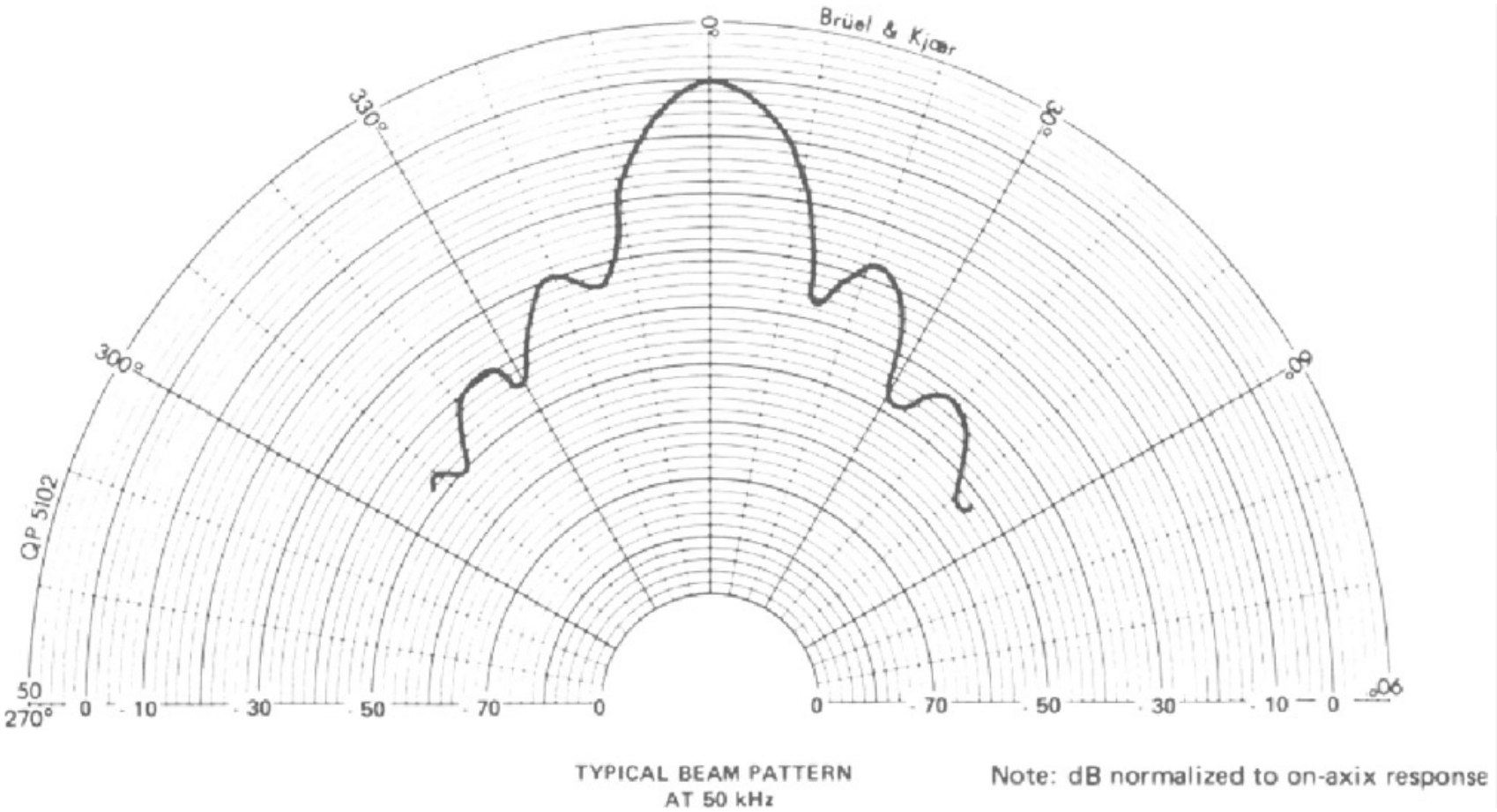


Polaroid ultrasonic transducer



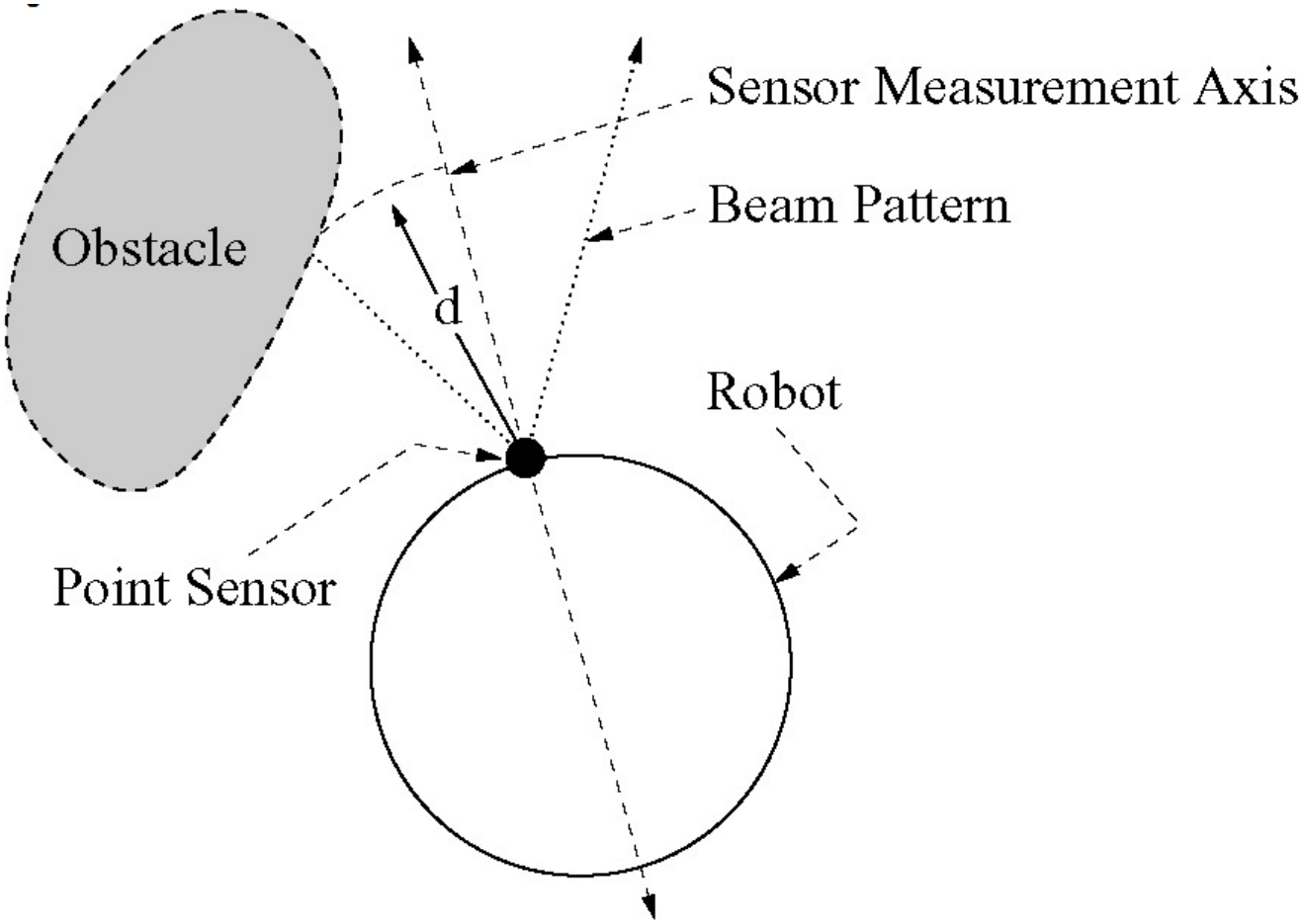
The disk on the right is the standard Polaroid ultrasonic transducer found on many mobile robots; the circuitry on the left drives the transducer.

Beam pattern for the Polaroid transducer.



This obstacle can be located anywhere along the angular spread of the sonar sensor's beam pattern. Therefore, the distance information that sonars provide is fairly accurate in depth, but not in azimuth.

Centerline model



The beam pattern can be approximated with a cone. For the commonly used Polaroid transducer, the arc base is 22.5 degrees